ASSESSMENT

Full-stack software engineer assessment

Take-home project: scalable document repository with metadata and search

Prepared by EEMW Digital Transformation Hub

The scenario

Your company needs a Document Repository where employees can upload, store, and search corporate documents (for example policies, reports, and technical docs). The system should be secure, scalable, and user-friendly.

You are asked to design and implement a Proof of Concept (POC) for this repository.

System design

Design a system for a scalable document repository that supports:

- Uploading and storing files
- Storing metadata (title, tags, permissions, versions) for each file
- Searching documents by title, tags, and uploader
- Viewing and downloading documents

Deliverable 1: a diagram of your system architecture, and a short description of your design choices.

Database design

Design a database schema that supports:

- Users with their department and role
- Documents with document tags. A document can have multiple tags, such as Finance and Legal
- Versions: a new version of a document replaces the old one, but the history is kept
- Permissions: some documents are restricted to certain departments

Proof of concept (POC)

Build a minimal working application with the following.

Backend (API). Implemented in Node.js and connected to the PostgreSQL database, with endpoints for:

- Upload document (store the file and its metadata)
- Search documents (by title, tags, uploader)
- View / download document
- Fetch the version history of a document

Frontend (UI). Implemented in Next.js, with the following features:

- Upload form (document and metadata)
- Search page (search bar and results list)
- Document detail page (view metadata, versions, and a download link)

Deliverable 2: a GitHub repository and a video of the POC working.

> **BONUS**
>
> Adding any ideas that are not mentioned in this document to enhance the system is a plus.

Tech stack

Layer	Technology
Backend	Node.js (Express, NestJS, or similar)
Frontend	Next.js (App Router)
Database	PostgreSQL (primary), Redis (caching, optional)
DevOps	Docker, Docker Compose, Git/GitHub

- Backend: a Node.js REST API exposing the endpoints listed above. Follow REST principles, and keep routes, business logic, and data access in separate layers.
- Frontend: Next.js (App Router) for the UI.
- Language: TypeScript preferred, plain JS is fine if you're more comfortable.
- Database: PostgreSQL as the primary database, with sensible keys, relationships, and indexes on the columns you search by. Redis for caching (for example search results) is a plus. Uploaded files can be stored on the local filesystem or in a Docker volume. No external hosted database needed.
- Docker: provide a docker-compose.yml that starts PostgreSQL (and Redis, if used) with a single command. Running the whole app through Docker Compose is a plus.
- Validation: basic checks on the API side too, not just in the forms (for example, reject an upload missing a required field or with a disallowed file type).

© 2026 EEMW Digital Transformation Hub. All rights reserved.

- Errors: sensible HTTP status codes and error responses on failure, not just the happy path (for example, 400 for invalid input, 404 for an unknown document).
- Styling: your choice (Tailwind, CSS Modules, styled-components, plain CSS). Pick one and be consistent.
- Organization: keep components and API routes reasonably organized rather than one giant file.

How to submit

1. Create a new GitHub repository for this project (public, or private with us added as collaborators, whichever you're comfortable with).
2. Push your project to it, with normal commit history. A handful of commits showing your progress is more useful to us than one giant commit.
3. Add a README.md at the root with a one-paragraph summary of what you built and any assumptions you made, how to run the backend and the frontend locally, the Node.js version you used (if it matters), and anything you'd do next with more time.
4. Include your architecture diagram and design-choices write-up in the repository (in the README or a docs/ folder).
5. Send us the repository link and the link to your POC video once you're done.

Example run commands for the README:

- Database: docker compose up -d
- Backend: cd backend && npm install && npm run dev
- Frontend: cd frontend && npm install && npm run dev
- Then open http://localhost:3000

© 2026 EEMW Digital Transformation Hub. All rights reserved.